DS 592 · INTRODUCTION TO SEQUENTIAL DECISION MAKING

Two-Level Contextual Bandits for Nurse Rostering

A Construct-and-Repair Framework and Its Regime of Applicability

Ria Sonalker · Achuthan Rathinam · Jihyeon Yun

April 2026

INRC-II · LinUCB · Construct-and-Repair

Nurse scheduling Problem setup

WHY IT MATTERS

- NP-hard scheduling problem
- Assign nurses across multiple weeks

Existing approaches:

- Exact Solver methods → high quality, slow
- Heuristics Solver methohds→ fast, not adaptive

CORE RESEARCH QUESTION

Can a learned bandit policy:

- select the right construction heuristic each week
- select the right repair operator each round

→ Minimize total penalty (INRC-II)

PRE DEFINED CONSTRAINTS (HARD AND SOFT)

- No single best heuristic
- Performance depends on schedule state

x Hard Constraints: Minimum staffing coverage, Forbidden shift successions, assigning based on nurse skill, Nurse availability and requested days off

7 number of soft constraints such as Actual coverage should meet the target/optimal level, Shift-off requests should be honored , total worked weekends shouldn't exceed nurse cap

Baseline ignores variation
→ Can adaptive policies do better?

Why nurse rostering? Why bandits?

WHY IT MATTERS

- NP-hard scheduling problem
- Assign nurses across multiple weeks

Existing approaches:

- Exact Solver methods → high quality, slow
- Heuristics Solver methohds→ fast, not adaptive

CORE RESEARCH QUESTION

Can a learned bandit policy:

- select the right construction heuristic each week
- select the right repair operator each round

→ Minimize total penalty (INRC-II)

THE DIFFICULTY

- No single best heuristic
- Performance depends on schedule state

Random selection → inefficient

4
construction arms
(week level)

→

18
repair operators
(repair level)

Baseline ignores variation
→ Can adaptive policies do better?

KEY IDEA: TWO-LEVEL CONTEXTUAL BANDIT

Using context to select the right action and defining penalty

Week-level Bandit

Week-Level Bandit

- 4 arms based on weekly construction heuristics/priority strategy
- One strategy picked per week

Context:

- workload, fatigue, demand

Goal:

- → build schedule week-by-week

Repair-level Bandit

Repair-Level Bandit

- 18 arms, or repair strategies
- ~500 rounds

Context:

- penalty distribution (key)

Goal:

- → iteratively reduce violations

Objective: Minimize total penalty

Let $v_i(\sigma, h_1)$ = number of violations of soft constraint i under schedule σ and initial history h_1, and let w_i be its published INRC-II weight.

Total penalty:
$P(\sigma, h_1) = w_1 \cdot v_1(\sigma,h_1) + w_2 \cdot v_2(\sigma,h_1) + \ldots + w_7 \cdot v_7(\sigma,h_1)$

or more compactly:
$P(\sigma, h_1) = \Sigma_{i=1}^{7} w_i \cdot v_i(\sigma, h_1)$

KEY IDEA: TWO-LEVEL CONTEXTUAL BANDIT

Contextual multi-armed bandits & LinUCB

Week-level Bandit

Week-Level Bandit

- 4 construction heuristics
- One decision per week

Context:

- workload, fatigue, demand

Goal:

- → build schedule week-by-week

Repair-level Bandit

Repair-Level Bandit

- 18 repair operators
- ~500 rounds

Context:

- penalty distribution (key)

Goal:

- → iteratively reduce violations

Shared Objective:
→ Minimize total penalty

Key Idea:
Use context to select the right action at each stage

KEY IDEA: TWO-LEVEL CONTEXTUAL BANDIT

Defining penalty reduction as reward

WEEK-LEVEL BANDIT

Each week $t \in \{1, \ldots, W\}$, the learner observes context $x^{(w)}_t \in \mathbb{R}^{d_w}$ (demand, carry-over history), selects arm $a^{(w)}_t \in \{1, \ldots, K_w\}$ (a construction heuristic), and extends the partial schedule through week t.

Reward = scaled penalty reduction from adding week t:

$$r^{(w)}_t = [\, P(\sigma_{\le t-1}, h_1) - P(\sigma_{\le t}, h_1) \,] / \rho_w$$

A positive reward means the chosen heuristic reduced the overall penalty. $\rho_w > 0$ is a reward scale factor.

REPAIR-LEVEL BANDIT

After construction, a repair loop runs for up to T rounds. Each round $j \in \{1, \ldots, T\}$, the learner observes context $x^{(r)}_j \in \mathbb{R}^{d_r}$ (current violations), selects arm $a^{(r)}_j \in \{1, \ldots, K_r\}$ (a local repair operator), and mutates the schedule while preserving hard feasibility.

Reward = one-step penalty reduction:

$$r^{(r)}_j = [\, P(\sigma_{j-1}, h_1) - P(\sigma_j, h_1) \,] / \rho_r$$

$r^{(r)}_j$ can be negative if the repair increases the soft penalty. $\rho_r > 0$ is a reward scale factor.

METHOD

Week level and repair level context vector construction

WEEK

ARMS (Kw = 4)

The week-level bandit selects among 4 construction heuristics. Each arm iterates over nurses in an arm-specific priority order and greedily fills the week's coverage requirements subject to hard feasibility. Arms differ only in their nurse-priority key:

1. coverage_first — lowest-penalty nurse first
2. fatigue_aware — rested nurses first
3. weekend_balancing — equalizes weekends worked
4. preference_respecting — honors shift-off requests

CONTEXT — $x^{(w)}_{\square} \in \mathbb{R}^8$ (all features clipped to [0, 1])

x_1 = coverage_slack = clip(Σ μ(d,f,q) / (|N|·7) , 0, 1)

x_2 = mean_fatigue_ratio = (1/|N|) · $\Sigma_{\square}$ consecWork(n) / $C_{\square}^{max}$

x_3 = weekend_spread_ratio = clip(std_n(weekendsWorked(n)) / max_n weekendCap(n) , 0, 1)

x_4 = request_density = clip(|shiftOffRequests| / |N| , 0, 1)

x_5 = max_assignment_saturation = clip(max$_{\square}$ totalAssignments(n) / $u_{\square}$, 0, 1)

x_6 = week_position = w / (W − 1)

REWARD

$r^{(w)}_{\square} = P(\sigma \le t_{-1}, h_1) - P(\sigma \le t, h_1)$

scaled by ρw and fed to the LinUCB update.

REPAIR

ARMS (Kr = 18)

Each operator exposes a find_violations routine (returns active violations it can address) and an apply routine (attempts a single feasibility-preserving edit).

Grouped by target constraint:

S1 — Coverage (4): pull_off_day_nurse · move_same_day_surplus_to_deficit · pull_from_adjacent_day_surplus · skill_reassignment

S2 — Consecutive work/shift (6): break_long_work_streak_mid · break_long_work_streak_start · break_long_work_streak_end · extend_short_work_streak · break_long_same_shift_streak · extend_short_same_shift_streak

S3 — Days off (3): break_long_rest_mid · break_long_rest_boundary · extend_short_rest

S4 — Preferences (1): swap_off_unwanted_shift

S5/S7 — Weekends (2): reduce_overloaded_weekends · fix_incomplete_weekend

S6 — Total assignments (1): rebalance_overworked_nurse

CONTEXT — $x^{(r)}_{\square} \in \mathbb{R}^7$

Let $P_{\square} = P(\sigma_{\square}, h_1)$ and $P_{\square}^{(s_i)}$ = penalty contribution from constraint S_i. One example:

$x_1 = P_{\square}^{(s1)} / \max(P_{\square}, 1)$

REWARD

$r^{(r)}_{\square} = P(\sigma_{\square-1}, h_1) - P(\sigma_{\square}, h_1)$

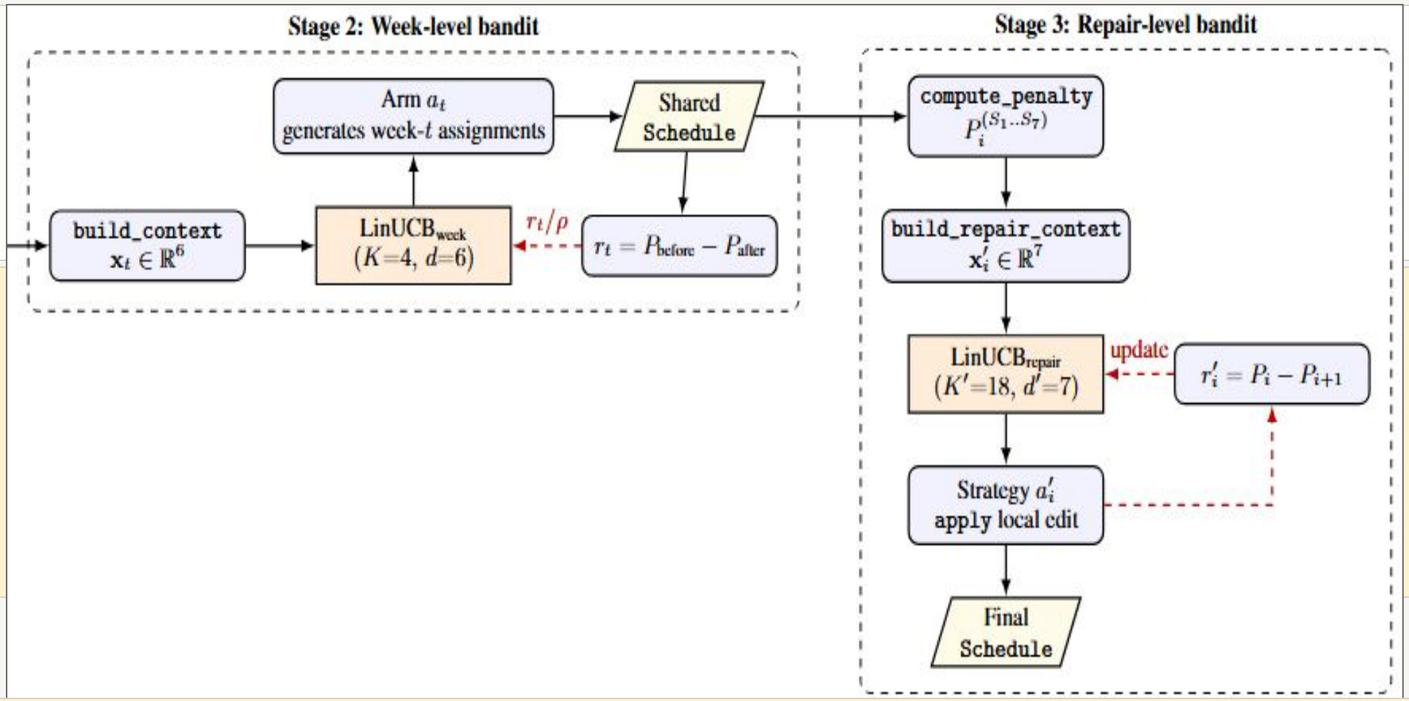
scaled by ρr. Note $r^{(r)}_{\square}$ can be negative if the action increases the soft penalty.

METHOD

The overall bandit pipeline

Key Idea: Choose actions based on current schedule state

Baseline	UCB1, ε-greedy, Thompson → ignore context → same action
LinUCB ★ (Contextual)	• Use state (context) • Learns reward per action Score = Exploitation + Exploration



Training protocol:

Warm Start

- First W_warm pulls cycle all arms uniformly, then LinUCB takes over
- Prevents early collapse onto a single arm
- W_warm = 8 (week) · 36 (repair)

Hyperparameters

- α = 1.0 · ρw = 1000 · ρr = 50 · T = 500

Initializer

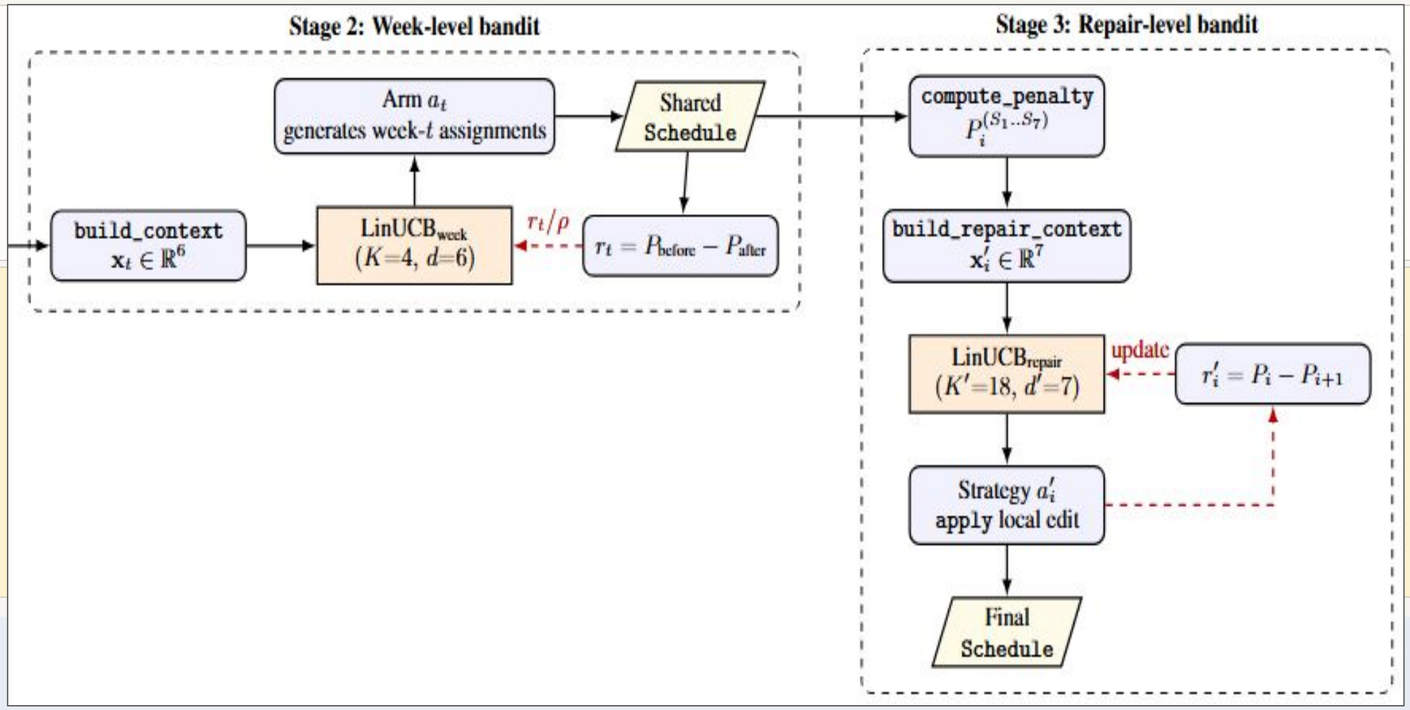
- Trained on greedy initializer schedules to match deployment distribution
- Branch-and-price solutions used only as OOD evaluation probe

METHOD

Why LinUCB?

Key Idea: Choose actions based on current schedule state

Baseline	UCB1, ε-greedy, Thompson → ignore context → same action
LinUCB ★ (Contextual)	• Use state (context) • Learns reward per action Score = Exploitation + Exploration



Assumption: Reward ≈ linear in context

PROBLEM SETUP

Two-level bandit

Actions	Context	Reward
• Week-Level: 4 construction heuristics • Repair-Level: 18 repair operators	• Week: workload, fatigue, demand • Repair: penalty distribution (key)	• Decrease in total penalty • Can be negative (repair step)

At each step: observe → act → update

Training
- Across multiple instances
- Warm-statrt to avoid early collapse

RESULTS

Week and Repair combinations study

BEST COMBINATION

UCB1 (week) + LinUCB (repair)
→ Lowest penalty: ~7,978

SUPPORTING POINTS

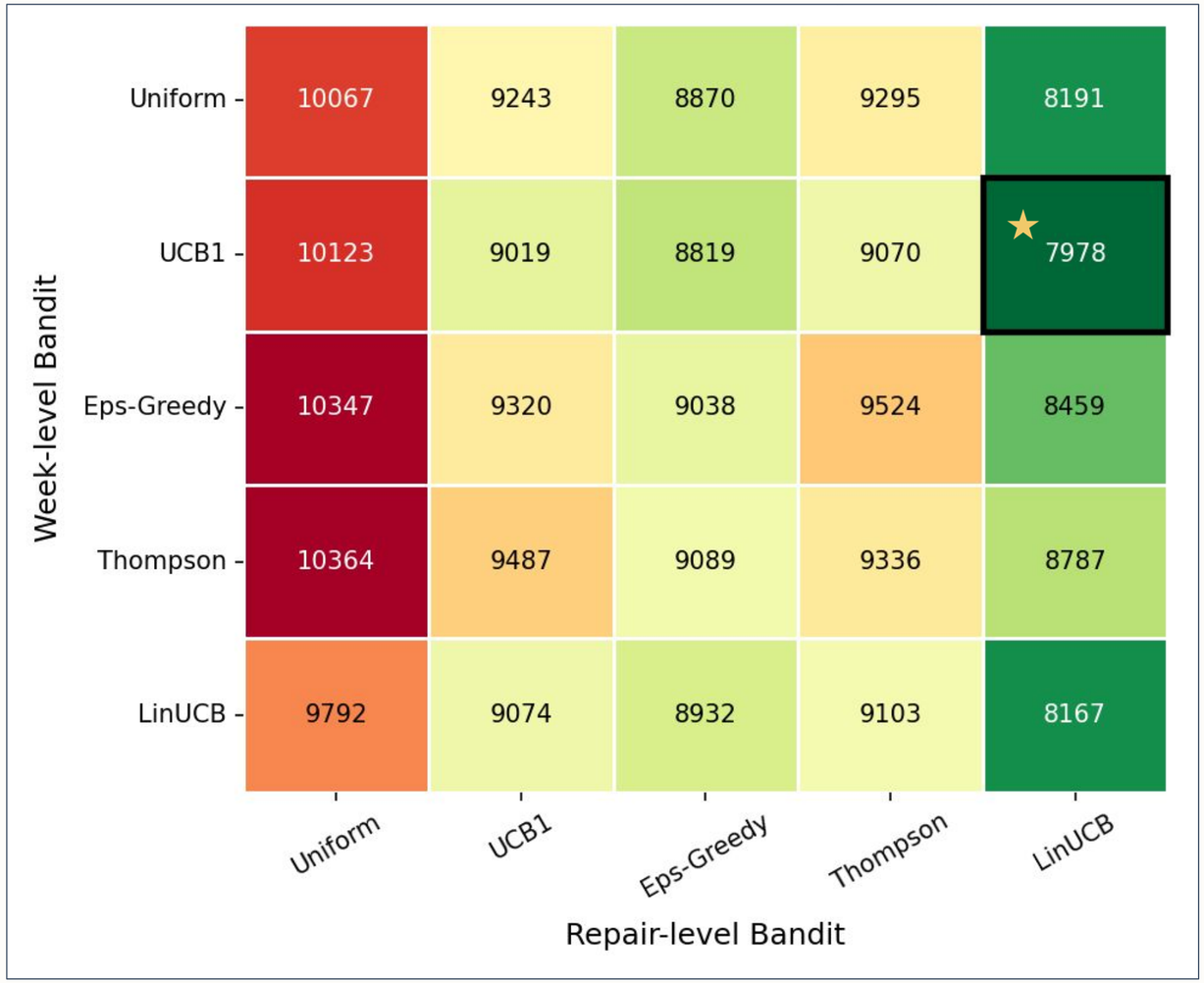
Key Findings:

- LinUCB repair = best across all settings
- Works regardless of construction method
- Uniform repair =worst

INSIGHT

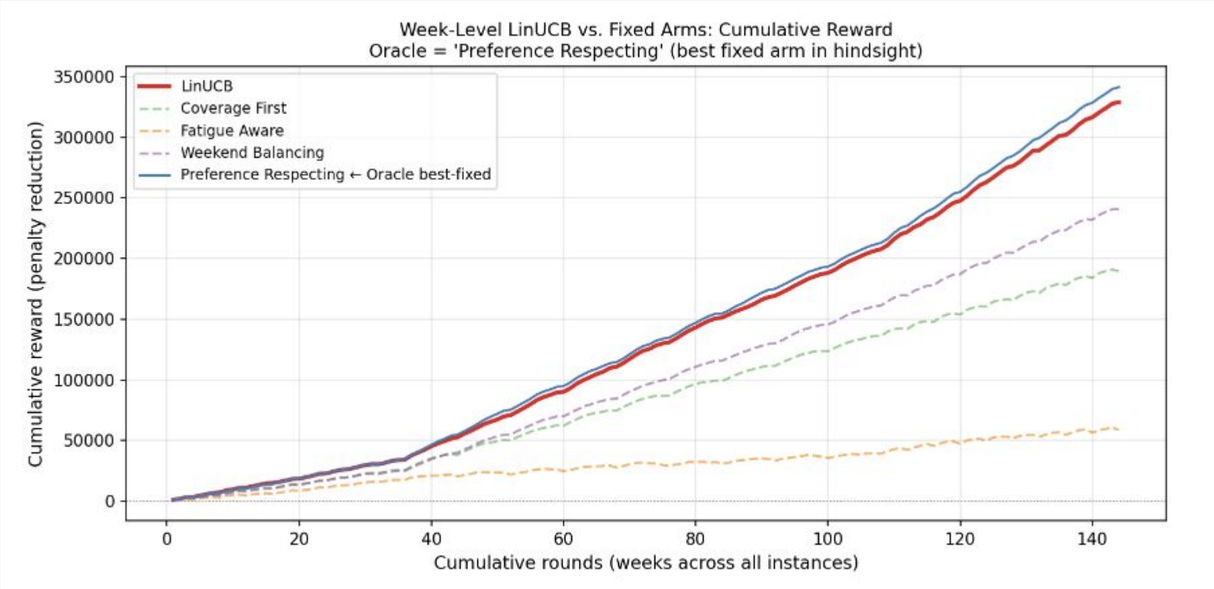
Why?

- Uses schedule context
- Selects relevant repair operator
- Avoids wasted actions



RESULTS

Does Context Help at the Week Level?



Week-level bandit makes only **one decision per week** → few pulls per instance
Compared LinUCB construction against **fixed-arm policies** (single heuristic used every week)
LinUCB tracks the **oracle best-fixed arm** closely in cumulative reward
Adaptivity provides **limited additional benefit** beyond simply picking a strong construction heuristic
Explains why non-contextual week-level methods (UCB1) remain competitive in the end-to-end comparison

RESULTS

Regime applicability // context ablation study

Limitations of our methods and future qork

Assumption violation

[add more]

KEY INSIGHT

When does context help?

Repair	vs	Construction
18 arms	action space	4 arms
500 rounds	decision round	few rounds
context helps	helpful	limited gain

Context matterms more in repair than construction.

Bandits work — until the schedule is nearly optimal

MAIN FINDING

Bandits improve scheduling
Especially effective in repair stage

Best:

UCB1 + LinUCB → ~7,978

LIMITATIONS

- State-dependent rewards
- Few week-level decisions
- Degrades near optimal

FINAL INSIGHT

Bandits work well in early / mid stages
but fail near optimality

→ Need sequential model (MDP / RL)

Thank you!